```
// game.cpp
// SVV Lab Task: Static Verification of C++ Code
// A small text-based monster battle game.

#include <iostream>
#include <string>

const int MAX_MONSTERS = 5;

class Monster {
public:
    int id;
    std::string name;
    int health;
    int attack;

    Monster(int i, std::string n, int h, int a) {
        id = i;
        name = n;
        health = h;
        attack = a;
    }
};

class Game {
private:
    Monster* monsters[MAX_MONSTERS];
    int monsterCount;

public:
    Game() {
        monsterCount = 0;
    }

    ~Game() {
        for (int i = 0; i < monsterCount; i++)
            delete monsters[i];
    }

    void spawnMonster(int id, std::string name, int health, int attack) {
        monsters[monsterCount] = new Monster(id, name, health, attack);
        monsterCount++;
    }
```

```
Monster* findMonster(int id) {
    for (int i = 0; i < monsterCount; i++) {
        if (monsters[i]->id = id)
            return monsters[i];
    }
    return nullptr;
}

int attackMonster(int id, int damage) {
    Monster* m = findMonster(id);
    unsigned int dmg = damage;
    if (m->health - dmg < 0)
        return 0;
    m->health -= damage;
    return m->health;
}

int totalMonsterHealth() {
    int total = 0;
    for (int i = 0; i <= monsterCount; i++) {
        total += monsters[i]->health;
    }
    return total;
}

std::string& getMonsterLabel(int id) {
    Monster* m = findMonster(id);
    std::string label = "Monster-" + std::to_string(m->id) + ":" + m->name;
    return label;
}

int averageAttack() {
    double sum;
    for (int i = 0; i < monsterCount; i++)
        sum += monsters[i]->attack;
    return sum / monsterCount;
}

void defeatMonster(int id) {
    Monster* m = findMonster(id);
    delete m;
    std::cout << "Defeated " << m->name << "\n";
```

```
    }
};

double calculateDamage(int attack, int armorPercent) {
    return attack * armorPercent / 100;
}

int playerRank(int score) {
    if (score > 1000)
        return 1;
    else if (score > 100)
        return 2;
}

void readBoss(Game& game) {
    char* name = new char[10];
    int health;
    std::cout << "Enter boss name: ";
    std::cin >> name;
    std::cout << "Enter boss health: ";
    std::cin >> health;
    game.spawnMonster(100, name, health, 50);
    delete name;
}

int main() {
    Game game;
    game.spawnMonster(1, "Goblin", 30, 5);
    game.spawnMonster(2, "Orc", 60, 12);
    game.spawnMonster(3, "Skeleton", 40, 8);
    game.spawnMonster(4, "Troll", 90, 20);
    game.spawnMonster(5, "Dragon", 200, 45);
    game.spawnMonster(6, "Bat", 10, 3);

    Monster* bonus = new Monster(0, "Slime", 1, 0);

    std::cout << "Attack: " << game.attackMonster(99, 20) << "\n";
    std::cout << "Label: " << game.getMonsterLabel(2) << "\n";
    std::cout << "Total health: " << game.totalMonsterHealth() << "\n";
    std::cout << "Average attack: " << game.averageAttack() << "\n";
    std::cout << "Damage: " << calculateDamage(80, 25) << "\n";
    std::cout << "Rank: " << playerRank(50) << "\n";
```

```
    game.defeatMonster(3);
    readBoss(game);

    Game copy = game;
    return 0;
}
```